UNIVERSIDAD TECNOLÓGICA DEL PERÚ

ARQUITECTURA ORIENTADA AL SERVICIO

Entregable 2 — Hotel Turístico

Diseño de la arquitectura SOA

JW Marriott Hotel Lima

Plataforma de Reservas, Paquetes Turísticos y Tours

INTEGRANTES

Aponte Matos, Mario Ivan
Rosas Tamara Walter Alexander
Fernando Chava Mendoza
Mayco Mendoza Malca
Jaramillo Rojas Moisés Alonso
Tabaco Perez, Luis Jayson

CURSO: Arquitectura Orientada al Servicio
SECCIÓN: 36321
DOCENTE: Lázaro Gómez, Jack Paul

Lima — Perú · 2026

1. Arquitectura general

El presente entregable traduce el análisis del Entregable 1 en un diseño arquitectónico concreto. Partiendo de los siete servicios identificados (Clientes, Habitaciones, Reservas, Paquetes, Tours, Pagos y Notificaciones), se define una arquitectura orientada a servicios en la que cada servicio es autónomo, expone un contrato REST y administra su propia base de datos, evitando el acoplamiento por datos compartidos.

El estilo adoptado es una SOA de servicios finamente granulados con un **API Gateway** como punto único de entrada, responsable del enrutamiento, la terminación TLS (HTTPS) y la autenticación/autorización mediante tokens JWT. El servicio **Reservas** actúa como servicio de proceso (orquestador): compone a los servicios de entidad y de utilidad para ejecutar el flujo de reserva integrada de extremo a extremo (Erl, 2007).

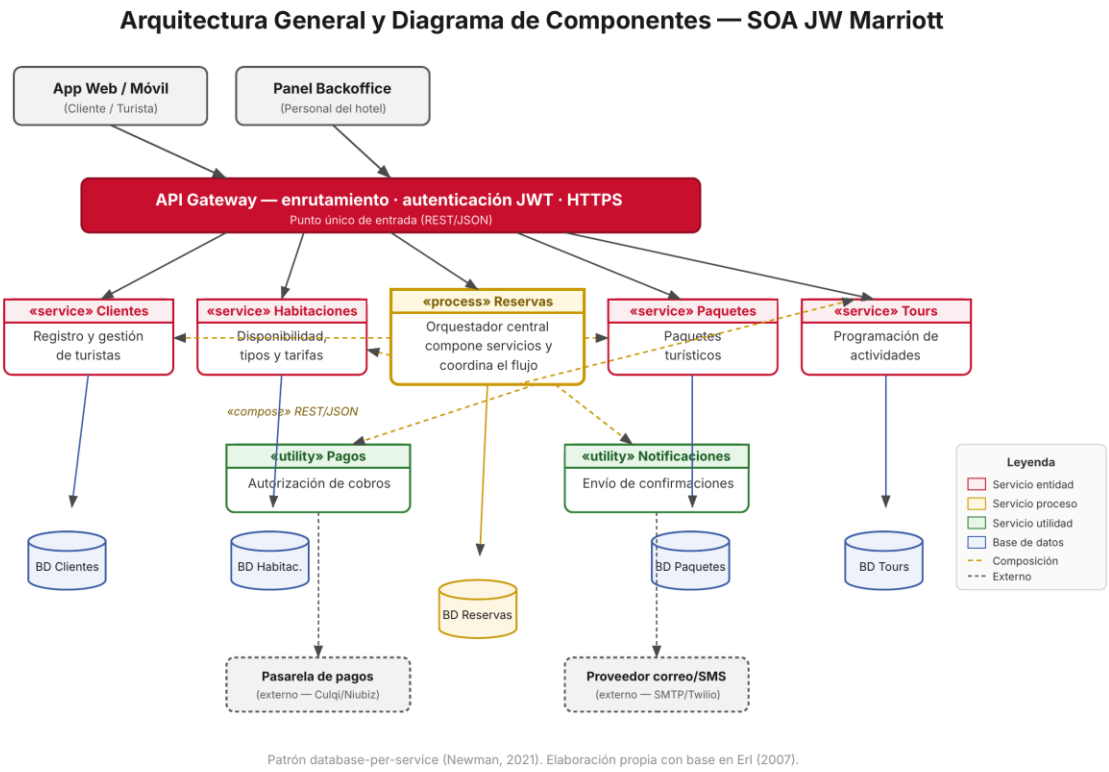


Figura 1. Arquitectura general y diagrama de componentes. Elaboración propia.

Capas de la arquitectura

- **Capa de presentación:** aplicación web/móvil del cliente y panel de backoffice del personal del hotel.
- **Capa de mediación:** API Gateway (enrutamiento, seguridad y exposición unificada de las APIs).
- **Capa de servicios:** servicios de entidad (Clientes, Habitaciones, Paquetes, Tours), de proceso (Reservas) y de utilidad (Pagos, Notificaciones).

- **Capa de persistencia:** una base de datos independiente por servicio (database-per-service).
- **Sistemas externos:** pasarela de pagos y proveedor de correo/SMS, integrados por API.

---

2. Definición de servicios y contratos

Cada servicio se define por su responsabilidad única, su interfaz (contrato) y su política de estado. Todos los servicios son stateless a nivel de aplicación: el estado conversacional no se mantiene en memoria del servicio, sino que se persiste en su base de datos o viaja en el token, lo que favorece la escalabilidad horizontal (Erl, 2007).

Servicio	Tipo	Contrato (recurso base)	Depende de
Clientes	Entidad	`/clientes`	—
Habitaciones	Entidad	`/habitaciones`	—
Reservas	Proceso	`/reservas`	Clientes, Habitaciones, Pagos, Notif.
Paquetes	Entidad	`/paquetes`	—
Tours	Entidad	`/tours`	—
Pagos	Utilidad	`/pagos`	Pasarela externa
Notificaciones	Utilidad	`/notificaciones`	Proveedor correo/SMS

Convenciones del contrato (aplican a todos los servicios):

- **Estilo:** REST sobre HTTPS; formato de intercambio JSON (UTF-8).
- **Autenticación:** token JWT en la cabecera Authorization: Bearer <token>.
- **Versionado:** prefijo de versión en la ruta (/api/v1).
- **Idempotencia:** GET, PUT y DELETE idempotentes; POST de reserva protegido con Idempotency-Key para evitar reservas duplicadas.

---

3. Diseño de APIs: endpoints, métodos, entradas y salidas

Se documentan los endpoints principales de cada servicio con su método HTTP, entrada (request), salida (response) y códigos de estado. Por extensión se detallan a nivel de contrato los servicios clave del flujo; el resto sigue el mismo patrón CRUD.

3.1 Servicio Clientes

Método	Endpoint	Descripción	Códigos
POST	`/api/v1/clientes`	Registra un cliente	201, 400, 409
GET	`/api/v1/clientes/{id}`	Obtiene un cliente	200, 404
PUT	`/api/v1/clientes/{id}`	Actualiza datos	200, 400, 404

Ejemplo — POST /api/v1/clientes

Request:

```
{
  "nombres": "Mario Iván",
  "apellidos": "Aponte Matos",
  "email": "mario@example.com",
  "documento": { "tipo": "DNI", "numero": "70000000" },
  "telefono": "+51 999888777"
}
```

Response 201 Created:

```
{
  "id": "CLI-000123",
  "nombres": "Mario Iván",
  "email": "mario@example.com",
  "estado": "ACTIVO",
  "creado_en": "2026-10-07T09:00:00-05:00"
}
```

3.2 Servicio Habitaciones

Método	Endpoint	Descripción	Códigos
GET	/api/v1/habitaciones?desde&hasta&tipo	Consulta disponibilidad	200, 400
GET	/api/v1/habitaciones/{id}	Detalle de habitación	200, 404
POST	/api/v1/habitaciones/{id}/bloqueo	Bloquea cupo (reserva)	200, 404, 409

Ejemplo — GET /api/v1/habitaciones?desde=2026-11-01&hasta=2026-11-04&tipo=SUITE

Response 200 OK:

```
{
  "disponibles": [
    {
      "id": "HAB-301",
      "tipo": "SUITE",
      "tarifa_noche": 920.00,
      "moneda": "PEN",
      "estado": "DISPONIBLE"
    }
  ],
  "total": 1
}
```

3.3 Servicio Reservas (orquestador)

Método	Endpoint	Descripción	Códigos
POST	/api/v1/reservas	Crea reserva integrada	201, 400, 402, 409
GET	/api/v1/reservas/{id}	Consulta una reserva	200, 404
DELETE	/api/v1/reservas/{id}	Cancela una reserva	200, 404, 409

Ejemplo — POST /api/v1/reservas (habitación + tour + pago)

Request (con cabecera Idempotency-Key):

```
{
  "cliente_id": "CLI-000123",
  "habitacion_id": "HAB-301",
  "desde": "2026-11-01",
  "hasta": "2026-11-04",
  "tour_id": "TOU-012",
  "paquete_id": null,
  "pago": { "metodo": "TARJETA", "token_tarjeta": "tok_abc123" }
}
```

Response 201 Created:

```
{
  "reserva_id": "RES-004521",
  "estado": "CONFIRMADA",
  "total": 3140.00,
  "moneda": "PEN",
  "items": ["HAB-301 x3 noches", "TOU-012"],
  "pago": { "estado": "APROBADO", "txn_id": "txn_99812" },
  "creado_en": "2026-10-07T09:05:00-05:00"
}
```

Response 409 Conflict (sin cupo) / 402 Payment Required (pago rechazado):

```
{
  "error": "SIN_DISPONIBILIDAD",
  "mensaje": "La habitación HAB-301 no tiene cupo en el rango solicitado.",
  "sugerencias": ["HAB-305", "HAB-410"]
}
```

3.4 Servicios Paquetes, Tours, Pagos y Notificaciones

Servicio	Método · Endpoint	Entrada → Salida
Paquetes	`GET /api/v1/paquetes`	filtros → lista de paquetes
Paquetes	`POST /api/v1/paquetes`	paquete → 201 paquete creado
Tours	`GET /api/v1/tours?fecha`	fecha → tours con cupo
Tours	`POST /api/v1/tours/{id}/reserva`	cliente, cupo → 201 / 409
Pagos	`POST /api/v1/pagos`	monto, token → 200 aprobado / 402
Notificaciones	`POST /api/v1/notificaciones`	destinatario, plantilla → 202 encolada

---

4. Modelo de datos

En coherencia con el principio de autonomía, cada servicio posee su propio esquema de base de datos; no existen claves foráneas entre esquemas. Las relaciones entre servicios se resuelven por referencia lógica (se guarda el identificador, no una FK física), y la consistencia del flujo la garantiza el orquestador Reservas (Newman, 2021).

Entidades principales por servicio

Servicio / Esquema	Entidad	Atributos principales
db_clientes	Cliente	id (PK), nombres, apellidos, email, doc_tipo, doc_numero, telefono, estado, creado_en
db_habitaciones	Habitacion	id (PK), tipo, piso, tarifa_noche, moneda, estado
db_habitaciones	Bloqueo	id (PK), habitacion_id, reserva_id, desde, hasta, estado
db_reservas	Reserva	id (PK), cliente_id, habitacion_id, paquete_id, tour_id, desde, hasta, total, estado, pago_txn_id, creado_en
db_paquetes	Paquete	id (PK), nombre, descripcion, precio, moneda, activo
db_tours	Tour	id (PK), nombre, fecha, cupo_total, cupo_disponible, precio, moneda
db_tours	ReservaTour	id (PK), tour_id, cliente_id, reserva_id, estado

Notas de diseño:

- Las referencias cruzadas (cliente_id, habitacion_id, tour_id, paquete_id) en db_reservas son identificadores lógicos hacia otros servicios, no claves foráneas físicas.
- **El estado de la Reserva sigue el ciclo:** PENDIENTE → CONFIRMADA → CANCELADA, controlando la coherencia del flujo integrado.
- El Bloqueo de habitación materializa la reserva temporal de cupo y previene la sobreventa (overbooking) y las reservas duplicadas.

---

5. Diagramas UML

5.1 Diagrama de componentes

El diagrama de componentes corresponde a la Figura 1 (sección 1), donde se muestran los servicios, sus interfaces REST, el API Gateway, las bases de datos por servicio y los sistemas externos.

5.2 Diagrama de secuencia

La Figura 2 muestra la interacción temporal entre componentes durante una reserva integrada, incluyendo el flujo alterno (sin cupo o pago rechazado) mediante un fragmento combinado alt.

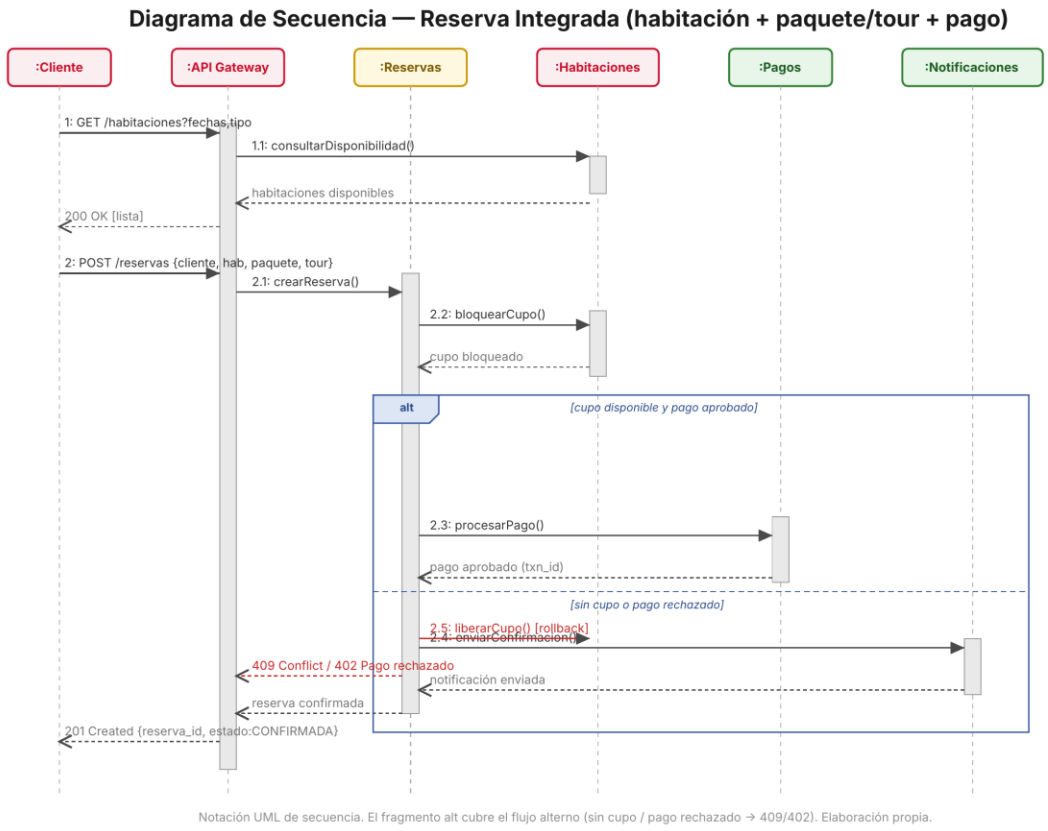


Figura 2. Diagrama de secuencia del flujo de reserva integrada. Elaboración propia.

5.3 Diagrama de despliegue

La Figura 3 representa la distribución física: el dispositivo del cliente, el servidor de aplicaciones con Docker donde cada servicio corre como contenedor independiente, el motor de base de datos y los sistemas externos.

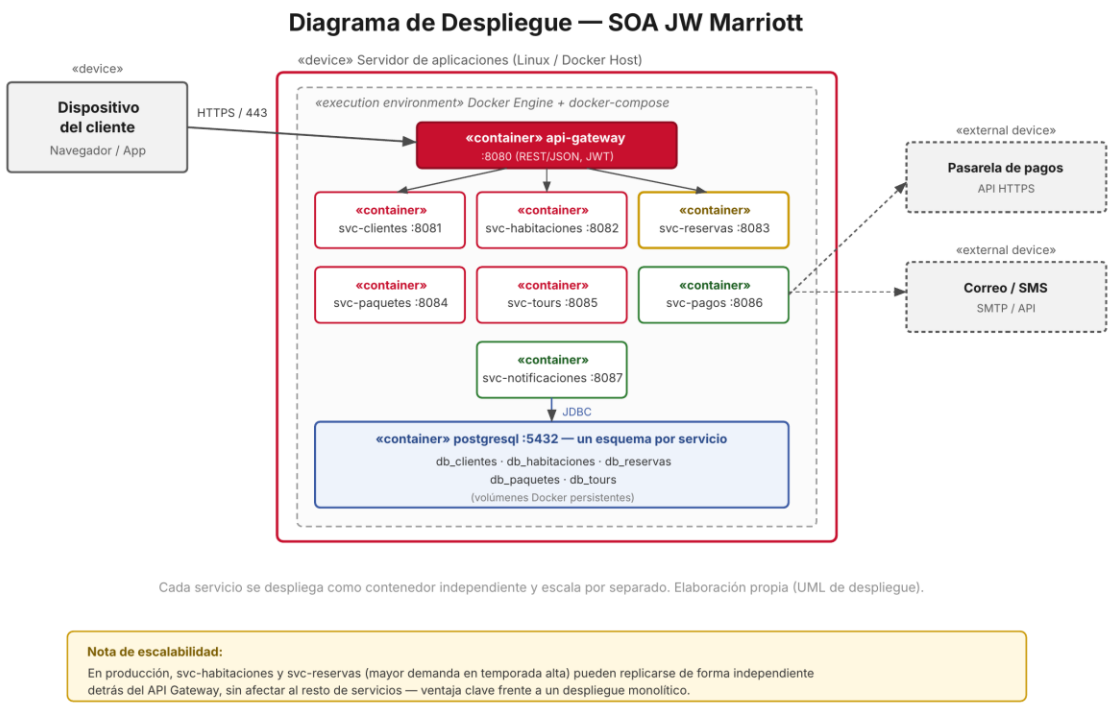


Figura 3. Diagrama de despliegue. Elaboración propia.

---

6. Justificación de tecnologías y decisiones arquitectónicas

Decisión	Opción elegida	Justificación
Lenguaje / framework de servicios	Java + Spring Boot	Madurez para REST, amplia adopción empresarial y soporte nativo de Spring Security (JWT) y Spring Data.
Estilo de comunicación	REST / JSON sobre HTTPS	Interoperable, simple de probar (Postman) y alineado con la sumilla del curso; menor acoplamiento que RPC binario.
Mediación	API Gateway	Punto único de entrada: centraliza seguridad, enrutamiento y versionado; desacopla al cliente de la topología interna.
Base de datos	PostgreSQL (una por servicio)	Relacional, robusta y gratuita; el aislamiento por esquema preserva la autonomía del servicio.
Persistencia de estado	Stateless + JWT	Permite escalar horizontalmente los servicios sin sesiones en memoria.
Empaquetado / despliegue	Docker + docker-compose	Contenedores independientes: cada servicio se construye, versiona y escala por separado.
Pruebas	Postman / colecciones	Evidencia funcional de las APIs exigida por el proyecto.
Control de versiones	Git / GitHub	Historial de trabajo y colaboración del equipo.

Ventaja frente al monolito. La separación en servicios autónomos permite desplegar, actualizar y escalar cada capacidad de forma independiente (por ejemplo, replicar Habitaciones y Reservas en temporada alta), aislar fallos y reutilizar los servicios de utilidad Pagos y Notificaciones. Esta modularidad es la evidencia central de la aplicación de SOA solicitada por el proyecto (Erl, 2007; Newman, 2021).

---

7. Referencias

Erl, T. (2007). *SOA: Principles of service design*. Prentice Hall.

MacKenzie, C. M., Laskey, K., McCabe, F., Brown, P. F., & Metz, R. (2006). *Reference model for service oriented architecture 1.0*. OASIS. https://docs.oasis-open.org/soa-rm/v1.0/soa-rm.pdf

Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2.ª ed.). O'Reilly Media.